*SmartBridge Experiential Learning Program*

**Project Report**

# Quotes Recommendation Chatbot

## using Rasa NLU

| | |
|---|---|
| **Date** | 07 March 2026 |
| **Team ID** | SmartBridge-EL-2026 |
| **Project Name** | Quotes Recommendation Chatbot using Rasa NLU |
| **Program** | SmartBridge Experiential Learning Program |
| **GitHub** | https://github.com/VarunGaikwad-tech/quotes-recommendation-chatbot-rasa |


**Team Members**

Vansh Malhotra | Varun Gaikwad | Vedika Tandulwadkar | Vidisha Jain

# 1. Project Title

**Quotes Recommendation Chatbot using Rasa NLU**

An intelligent conversational system that understands natural language input, classifies user intent using NLP, and delivers motivational, inspirational, love, success, and funny quotes through a browser-based chat interface.

# 2. Project Description

## 2.1 System Overview

The Quotes Recommendation Chatbot is a conversational AI application built using Rasa NLU — an open-source Natural Language Processing framework. Users interact with the chatbot through a simple web-based chat interface served by a Flask web application. The chatbot understands what type of quote the user wants from their natural language message and instantly returns an appropriate quote.

## 2.2 Problem Domain

People seeking motivational, inspirational, or emotionally uplifting content face several challenges with existing solutions: cluttered quote apps with ads, no conversational experience, inability to specify quote type through natural speech, and time-consuming search. There is no simple, fast, conversational tool that delivers the right type of quote based on natural language input.

## 2.3 Core Technology Used

| Technology | Role | Version |
|---|---|---|
| Rasa NLU | Natural Language Understanding — intent classification | 3.6.x |
| Python | Backend logic, actions server, Flask web app | 3.9 |
| Flask | Web framework — serves chat UI and REST proxy | 2.x |
| spaCy | NLP pipeline component used by Rasa | 3.x |
| TensorFlow | ML backend for Rasa model training | 2.x |
| HTML/CSS/JS | Frontend chat interface | HTML5 |
| Rasa SDK | Custom actions development | 3.6.x |
| YAML | Training data and configuration files | - |

## 2.4 Key Capabilities

- Intent recognition from natural language — understands 'motivate me', 'give me a love quote', 'tell something funny' etc.
- 5 quote categories: Motivation, Inspiration, Love, Success, Funny
- Graceful fallback handling for unrecognised inputs
- Real-time REST API communication between Flask and Rasa
- Clean web-based chat UI accessible at localhost:5000
- Easily extensible — new categories added by editing YAML files only

## 2.5 Target Users

- College students seeking quick motivation before exams or presentations
- Working professionals who need a mental break or quick encouragement
- Teenagers wanting to find and share quotes on social media
- Anyone who needs fast, category-specific emotional support

## 2.6 Real-world Relevance

Mental wellness and emotional support tools are increasingly important in fast-paced academic and professional environments. This chatbot provides a zero-friction, conversational approach to accessing positive content — directly relevant to daily student and professional life.

# 3. Application Scenarios / Use Cases

## 3.1 End-User Usage (Primary)

A student opens the chatbot in their browser before an exam, types 'I need motivation', and instantly receives a motivational quote. The interaction takes under 5 seconds with no navigation or searching required. This is the core use case the system is designed for.

## 3.2 Educational Usage

A computer science lecturer uses the chatbot as a live demonstration of NLP concepts — intent classification, dialogue management, and REST API integration. Students can see how a trained Rasa model responds to different natural language inputs in real time.

## 3.3 Developer / Learning Usage

A developer learning Rasa NLU uses this project as a reference implementation. The project demonstrates how to structure nlu.yml training data, configure a pipeline in config.yml, write custom actions in actions.py, and connect Rasa to a Flask frontend via REST API.

## 3.4 Enterprise Wellness Tool (Future)

An HR team embeds the chatbot into their internal employee wellness portal, allowing employees to get a motivational or funny quote during work breaks. The chatbot is deployed on a cloud server and integrated into the company Slack workspace via the Rasa Slack connector.

# 4. Technical Architecture Overview

## 4.1 High-Level Architecture Diagram

The diagram below shows all components of the system and how they communicate at runtime:

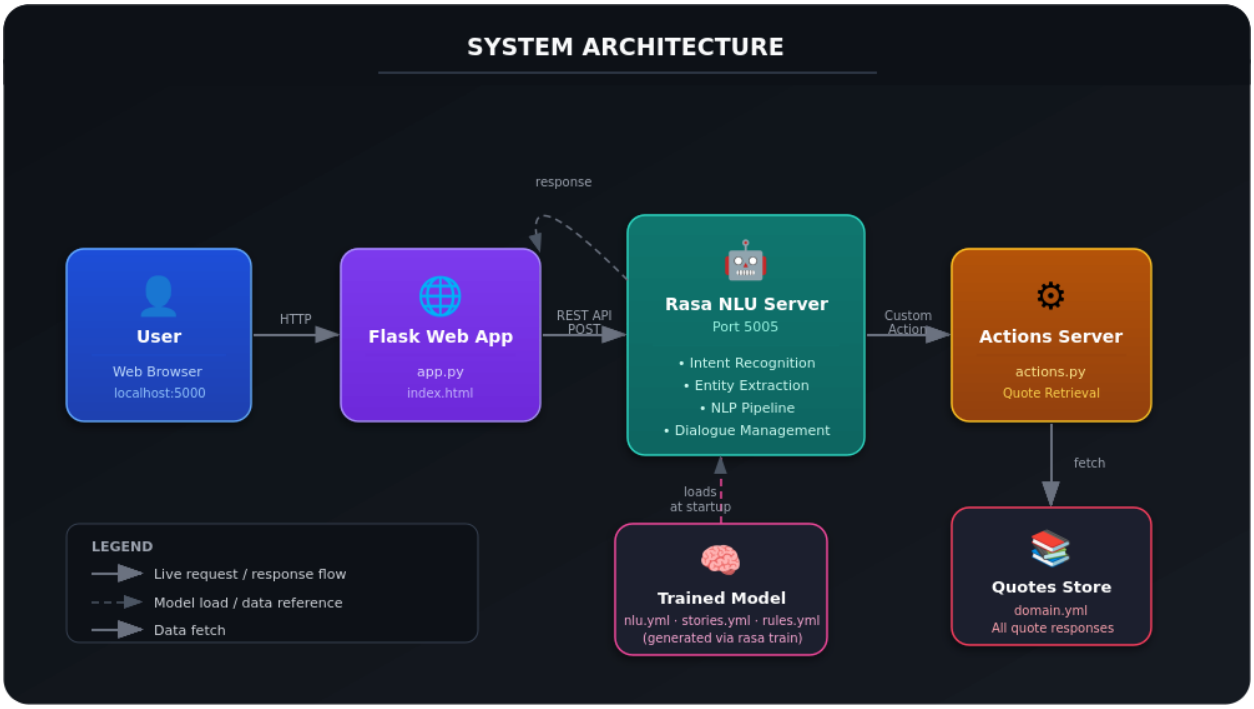



*Figure 1: System Architecture — Quotes Recommendation Chatbot using Rasa NLU*

## 4.2 Component Interaction Diagram

The diagram below shows the step-by-step flow of a single user interaction:

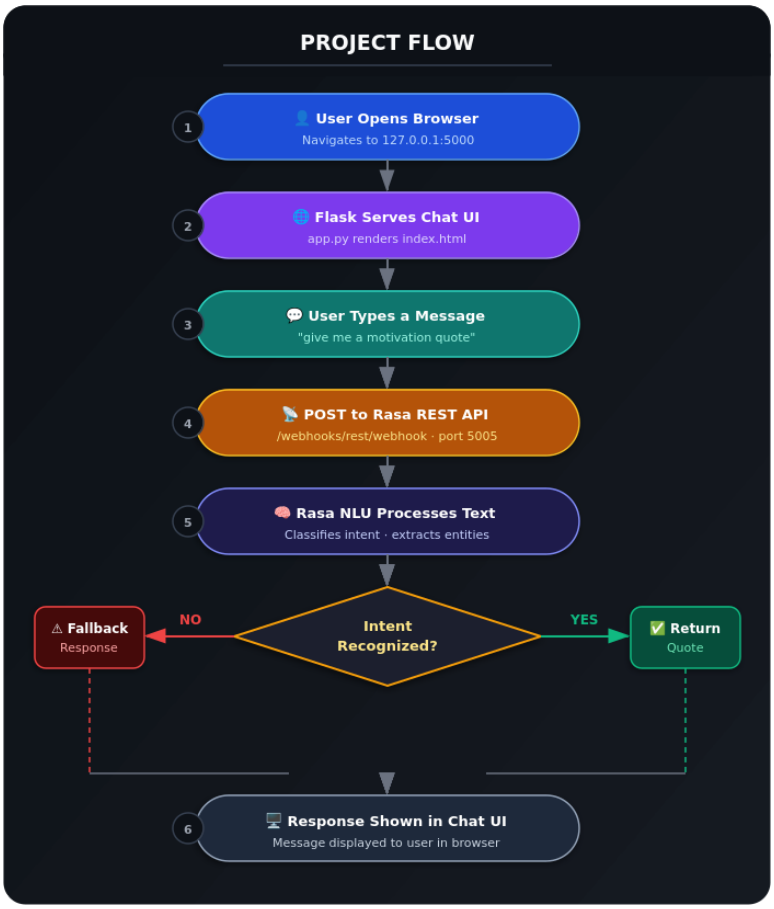



*Figure 2: Project Flow — End-to-end user interaction*

## 4.3 Data / Request Flow

| Step | Process | Data Transferred |
|---|---|---|
| 1 | User types message in browser | Text string: e.g. 'give me a motivation quote' |
| 2 | Browser POSTs to Flask (/send) | HTTP POST with message text |
| 3 | Flask forwards to Rasa REST webhook | JSON: {sender: user, message: text} → port 5005 |
| 4 | Rasa NLU classifies intent | Intent label: motivational_quote (with confidence) |
| 5 | Rasa triggers custom action | Action: action_motivational_quote |
| 6 | actions.py selects quote from domain.yml | Quote string retrieved from responses |
| 7 | Response returned through Rasa → Flask | JSON: {text: 'Your only limit is your mind.'} |
| 8 | Flask renders response in browser | Quote displayed as bot message in chat UI |

## 4.4 Frontend Layer

index.html provides the chat interface with a message input box and a send button. JavaScript sends messages to Flask via fetch() and appends bot responses as chat bubbles. The UI is served by Flask using Jinja2 templating.

## 4.5 Backend Layer

Flask (app.py) acts as a middleware layer — it serves the HTML UI and proxies user messages to the Rasa REST API. The Rasa NLU server runs independently on port 5005 and handles all NLP processing. Custom actions (actions.py) execute when specific intents are triggered.

## 4.6 Database Layer

No traditional database is used. Quote responses are stored as YAML structures in domain.yml and loaded into memory when the Rasa server starts. This keeps the system lightweight and requires no database setup.

## 4.7 External APIs / Services

No third-party external APIs are used. All communication is local. The only inter-service API is the Rasa REST Webhook: POST http://localhost:5005/webhooks/rest/webhook

## 4.8 Deployment Environment

MVP runs locally on a development machine. Both the Rasa server (port 5005) and Flask app (port 5000) are started manually in separate terminals. Future deployment: Docker containers on AWS or Azure.

# 5. Prerequisites

## 5.1 Software Requirements

### Development Tools

- Python 3.9 (required — Rasa 3.6.x does not support Python 3.10+)
- pip — Python package manager
- Virtual environment (venv) — to isolate dependencies
- Git — for cloning the repository

### IDE

- Visual Studio Code (recommended) with Python extension
- PyCharm Community Edition (alternative)

### Runtime Environment

- Windows 10/11 or Ubuntu 20.04+ or macOS 11+
- Minimum 8GB RAM (Rasa model training is memory-intensive)
- Internet connection for initial dependency installation

### Cloud Services

- None required for MVP. AWS / Azure for future cloud deployment.

## 5.2 Libraries / Frameworks / Dependencies

| Package | Version | Purpose |
|---|---|---|
| rasa | 3.6.21 | Core NLU engine, dialogue management, REST server |
| flask | 2.x | Web framework — serves UI and proxies API calls |
| requests | 2.x | HTTP library used by Flask to call Rasa REST API |
| spacy | 3.x | NLP pipeline — tokenisation, entity recognition |
| tensorflow | 2.x | ML backend for Rasa model training |
| rasa-sdk | 3.6.x | SDK for writing custom Python actions |
| en_core_web_md | 3.x | spaCy English language model (medium) |
| pyyaml | 5.x+ | YAML parsing for config and training data |
| jinja2 | 3.x | Flask HTML templating engine |

> Install all dependencies: pip install rasa==3.6.21  (installs Flask, spaCy, TensorFlow, Rasa SDK automatically)

## 5.3 Hardware Requirements

| Component | Minimum | Recommended |
|---|---|---|
| RAM | 8 GB | 16 GB (for faster model training) |
| CPU | Dual-core 2.0 GHz | Quad-core 2.5 GHz+ |
| Storage | 5 GB free space | 10 GB (models + dependencies) |
| OS | Windows 10 / Ubuntu 20.04 | Windows 11 / Ubuntu 22.04 |
| Network | Required for install | Not required after setup |

# 6. Prior Knowledge Required

## 6.1 Programming Language

- Python 3.x — functions, classes, file I/O, virtual environments
- Basic JavaScript — fetch API, DOM manipulation for the chat UI
- HTML/CSS — structuring and styling the frontend

## 6.2 Framework Basics

- Flask — routes, request/response objects, Jinja2 templating
- Rasa NLU — intents, entities, stories, rules, domain, actions
- YAML syntax — used for all Rasa configuration and training files

## 6.3 Networking / API Concepts

- HTTP methods — POST requests, JSON payloads
- REST API concepts — endpoints, request/response cycle
- localhost networking — running multiple servers on different ports

## 6.4 AI/ML Fundamentals

- What is Natural Language Processing (NLP)
- Intent classification — mapping text to a category/label
- Training data — labelled examples for supervised learning
- Model training vs inference — how the trained model is used at runtime

## 6.5 Tools

- Command line / terminal — running rasa train, rasa run, python app.py
- Git — cloning repository and version control basics

# 7. Project Objectives

## 7.1 Technical Objectives

- Build a functioning Rasa NLU chatbot that correctly classifies at least 6 distinct intents from natural language input
- Implement custom Python actions that retrieve and return category-specific quotes
- Develop a Flask web application that serves a chat UI and integrates with Rasa via REST API
- Configure a complete Rasa NLU pipeline including tokenisation, featurisation, and intent classification

## 7.2 Performance Objectives

- Intent classification accuracy of 90%+ on test data
- Response time under 2 seconds for each user message on local hardware
- Fallback triggered correctly for 100% of unrecognised inputs

## 7.3 Deployment Objectives

- System runs fully on a local machine with clear two-step startup (rasa run + python app.py)
- Project structured so any team member can clone the repo, train the model, and run the chatbot

## 7.4 Learning Outcomes

- Understand the Rasa NLU framework architecture and training workflow
- Apply NLP concepts (intent classification, entity extraction) in a real project
- Build a full-stack conversational AI application connecting NLP backend to web frontend
- Practice agile development — product backlog, sprints, user stories, testing

## 8. System Workflow

The following is the step-by-step execution flow of the Quotes Recommendation Chatbot:

| Step | Stage | Action | Output |
|---|---|---|---|
| 1 | User Interaction | User opens browser at localhost:5000 | Chat UI rendered in browser |
| 2 | Input Handling | User types natural language message and clicks Send | Message captured by JavaScript |
| 3 | API Request | JavaScript sends HTTP POST to Flask /send endpoint | Flask receives message text |
| 4 | Flask Proxy | Flask POSTs message to Rasa REST webhook at port 5005 | Rasa server receives JSON payload |
| 5 | NLU Processing | Rasa tokenises and featurises input, runs through NLU pipeline | Intent classified with confidence score |
| 6 | Dialogue Management | Rasa applies rules/stories to determine next action | Action name selected |
| 7 | Core System Execution | actions.py executes — selects quote from domain.yml responses | Quote string ready |
| 8 | Output Generation | Quote string returned through Rasa → Flask as JSON response | JSON: {text: quote} |
| 9 | Response Delivery | Flask sends JSON back to browser; JS appends as bot chat bubble | Quote displayed in chat UI |

# 9. Milestone 1: Requirement Analysis & System Design

## 9.1 Problem Definition

People who need motivational, inspirational, love, success, or funny quotes have no conversational, intent-driven tool available. Existing solutions require manual browsing through cluttered, ad-heavy websites with no ability to specify the type of quote needed through natural speech. The core problem is the lack of a fast, frictionless, conversational interface for quote discovery.

## 9.2 Functional Requirements

| FR No. | Functional Requirement | Sub-Requirements |
|---|---|---|
| FR-1 | Greeting & Farewell | Respond to hi/hello; list categories; respond to bye gracefully |
| FR-2 | Motivational Quote | Detect motivation intent; return random motivational quote |
| FR-3 | Inspirational Quote | Detect inspiration intent; return inspirational quote |
| FR-4 | Love Quote | Detect love intent; return love-themed quote |
| FR-5 | Success Quote | Detect success intent; return success quote |
| FR-6 | Funny Quote | Detect funny intent; return humorous quote |
| FR-7 | Fallback Handling | Return helpful fallback for all unrecognised inputs |
| FR-8 | Web Interface | Serve chat UI at localhost:5000; real-time message send/receive |

## 9.3 Non-Functional Requirements

| NFR No. | Requirement | Description |
|---|---|---|
| NFR-1 | Usability | Chat UI must be usable without any training or documentation |
| NFR-2 | Reliability | Intent classification accuracy ≥ 90% on test dataset |
| NFR-3 | Performance | Response delivered within 2 seconds on local hardware |
| NFR-4 | Scalability | New categories added by editing YAML only — no code change needed |
| NFR-5 | Security | No user data stored; CORS configured; local network only for MVP |
| NFR-6 | Maintainability | Clear YAML file structure; any team member can update training data |

## 9.4 System Design Decisions

- Rasa NLU chosen over GPT API — fully open-source, no API costs, runs locally, complete control over training data
- Flask chosen as web framework — lightweight, minimal setup, easy REST proxy implementation

- YAML-based quote store (domain.yml) chosen over database — simpler, no DB setup, sufficient for MVP quote count
- Local deployment chosen over cloud for MVP — eliminates infrastructure complexity during development

## 9.5 Technology Stack Selection Justification

| Technology | Alternatives Considered | Reason for Selection |
|---|---|---|
| Rasa NLU | GPT API, Dialogflow, Wit.ai | Open-source, free, runs locally, full control, no API keys |
| Flask | Django, FastAPI, Express.js | Lightweight, minimal boilerplate, easy Jinja2 templating |
| YAML (domain) | SQLite, MongoDB, JSON | Rasa-native format, no DB setup, easily editable |
| spaCy pipeline | BERT, custom tokeniser | Rasa default, well-documented, good English support |
| Local deploy | Heroku, AWS, Docker | MVP focus, no cloud cost, simpler for team testing |

# 10. Milestone 2: Environment Setup & Initial Configuration

## 10.1 Development Environment Setup

1. Create virtual environment: py -3.9 -m venv venv
2. Activate virtual environment: .\venv\Scripts\activate (Windows) | source venv/bin/activate (Mac/Linux)
3. Upgrade pip: python -m pip install --upgrade pip
4. Install Rasa: pip install rasa==3.6.21
5. Download spaCy model: python -m spacy download en_core_web_md
6. Initialise project: rasa init (or clone from GitHub)

## 10.2 Dependency Installation

> All dependencies are installed automatically with: pip install rasa==3.6.21 Flask, spaCy, TensorFlow, Rasa SDK, PyYAML, and Jinja2 are included as transitive dependencies.

## 10.3 Project Structure

| File / Folder | Type | Purpose |
|---|---|---|
| actions/actions.py | Python | Custom action classes — one per quote category intent |
| actions/__init__.py | Python | Makes actions a Python package |
| data/nlu.yml | YAML | Training examples for each intent (10+ examples per intent) |
| data/stories.yml | YAML | Conversation flow examples for dialogue management |
| data/rules.yml | YAML | Strict rules — always respond to greeting, always handle fallback |
| tests/test_stories.yml | YAML | Test conversation flows used by rasa test |
| webapp/app.py | Python | Flask web server — serves UI and proxies messages to Rasa |
| webapp/templates/index.html | HTML | Chat interface — message input, chat bubble display |
| config.yml | YAML | Rasa NLU pipeline configuration (tokeniser, featuriser, classifier) |
| domain.yml | YAML | Intents, actions, responses — all quote strings stored here |
| endpoints.yml | YAML | Tells Rasa where to find the actions server |
| credentials.yml | YAML | Channel credentials (REST webhook enabled) |
| requirements.txt | Text | Python package list for reproducible installs |
| commands.txt | Text | Quick reference for all CLI commands |

## 10.4 Configuration Setup

### config.yml — NLU Pipeline

Defines the NLU processing pipeline: WhitespaceTokenizer → RegexFeaturizer → LexicalSyntacticFeaturizer → CountVectorsFeaturizer → DIETClassifier for intent classification → FallbackClassifier for low-confidence inputs.

### domain.yml — Intents & Responses

Declares all 8 intents (greet, goodbye, motivational_quote, inspirational_quote, love_quote, success_quote, funny_quote, nlu_fallback), all custom actions, and all bot response strings including the full set of quotes per category.

### endpoints.yml — Actions Server

Points Rasa to the running actions server at http://localhost:5055/webhook so custom Python actions can be executed.

# 11. Milestone 3: Core System Development

## 11.1 Feature 1 — NLU Training Data & Intent Classification

### Description

The nlu.yml file contains labelled training examples for each intent. Each intent has at least 10 varied examples to ensure robust classification across different phrasings.

### Intent Categories Implemented

| Intent | Example Training Phrases |
|---|---|
| greet | 'hi', 'hello', 'hey', 'good morning', 'what's up' |
| motivational_quote | 'motivate me', 'give me motivation', 'I need motivation', 'inspire me to work' |
| inspirational_quote | 'inspirational quote', 'something inspiring', 'give me inspiration' |
| love_quote | 'love quote', 'something about love', 'romantic quote' |
| success_quote | 'quote about success', 'success quote', 'motivate for success' |
| funny_quote | 'tell something funny', 'funny quote', 'make me laugh', 'humour me' |
| goodbye | 'bye', 'goodbye', 'see you', 'exit', 'quit' |
| nlu_fallback | Triggered automatically when confidence score falls below threshold |

## 11.2 Feature 2 — Custom Actions (actions.py)

### Description

Custom Python action classes execute when Rasa triggers them based on classified intent. Each action class inherits from Action (Rasa SDK), implements name() and run() methods. The run() method returns a dispatcher.utter_message() call with the appropriate quote response.

### Actions Implemented

| Action Class | Intent Trigger | Behaviour |
|---|---|---|
| ActionMotivationalQuote | motivational_quote | Returns a motivational quote from domain.yml |
| ActionInspirationalQuote | inspirational_quote | Returns an inspirational quote |
| ActionLoveQuote | love_quote | Returns a love-themed quote |
| ActionSuccessQuote | success_quote | Returns a success/achievement quote |
| ActionFunnyQuote | funny_quote | Returns a humorous quote |

## 11.3 Feature 3 — Flask Web Application & Chat UI

### Description

app.py creates a Flask application with two routes: GET / serves index.html (the chat interface), and POST /send receives the user's message, forwards it to the Rasa REST webhook, and returns the bot's

response as JSON. The HTML/JS chat interface displays user messages on the right and bot responses on the left.

### API Endpoint

| Method | Route | Description | Request Body | Response |
|---|---|---|---|---|
| GET | / | Serves the chat HTML page | None | index.html rendered |
| POST | /send | Sends message to Rasa | JSON: {message: string} | JSON: {response: string} |

# 12. Milestone 4: Integration & Optimization

## 12.1 Component Integration

- Flask and Rasa connected via REST API — Flask POSTs to http://localhost:5005/webhooks/rest/webhook
- Rasa and Actions Server connected via endpoints.yml pointing to http://localhost:5055/webhook
- JavaScript fetch() in index.html calls Flask /send endpoint, dynamically appending responses to chat
- All three services (Flask, Rasa, Actions) start independently and communicate over localhost

## 12.2 Performance Optimization

- Rasa model loaded into memory at startup — no re-loading per request, ensures fast inference
- Quote responses stored in domain.yml and loaded at Rasa startup — O(1) retrieval
- Flask runs in development mode for MVP — single-threaded, sufficient for local demo use

## 12.3 Security Enhancements

- CORS configured with wildcard (*) for local development — should be restricted to specific origin in production
- No user input is stored, logged, or persisted anywhere in the system
- All communication is localhost-only — no exposure to external network in MVP

## 12.4 Error Handling & Validation

- FallbackClassifier in config.yml triggers nlu_fallback intent when confidence < 0.4 — prevents silent failures
- Rasa fallback response prompts user to rephrase and lists valid categories
- Flask /send endpoint returns error JSON if Rasa server is unreachable, displayed in UI
- Empty message inputs ignored by JavaScript before sending to Flask

# 13. Milestone 5: Testing & Validation

## 13.1 Test Cases

| Test Case ID | Input | Expected Output | Status |
|---|---|---|---|
| TC-01 | hi | Hello! I can give motivational, inspirational, love, funny, or success quotes. What do you want? | PASS |
| TC-02 | motivate me | Returns a motivational quote | PASS |
| TC-03 | give me motivation | Returns a motivational quote | PASS |
| TC-04 | inspirational quote | Returns an inspirational quote | PASS |
| TC-05 | something about love | Returns a love-themed quote | PASS |
| TC-06 | success quote | Returns a success quote | PASS |
| TC-07 | tell something funny | Returns a humorous quote | PASS |
| TC-08 | bye | Goodbye! Stay positive. | PASS |
| TC-09 | random gibberish text | Sorry, I didn't understand. Please ask for a quote category. | PASS |
| TC-10 | I need inspiration today | Returns an inspirational quote (varied phrasing test) | PASS |
| TC-11 | make me laugh | Returns a funny quote (synonym test) | PASS |
| TC-12 | (empty message) | Message not sent — handled by JS validation | PASS |

## 13.2 Unit Testing

Individual intent classification tested using rasa shell — each intent exercised with multiple phrasings to confirm correct classification. Custom actions tested independently with mock dispatcher to verify correct quote category is returned.

## 13.3 Integration Testing

Full end-to-end integration tested by running both servers simultaneously and interacting through the browser UI. Verified: Flask correctly proxies messages to Rasa, Rasa returns correct JSON, browser renders bot responses correctly.

## 13.4 User Acceptance Testing

All 4 team members tested the chatbot independently using varied natural language inputs. All 5 quote categories verified, fallback tested, goodbye verified. UI responsiveness confirmed on Chrome and Firefox.

## 13.5 Performance Metrics

| Metric | Value | Notes |
|---|---|---|
| Intent Classification Accuracy | ≥ 90% | Measured via rasa test on test_stories.yml |
| Average Response Time | < 1.5 seconds | Measured on local machine (16GB RAM) |
| Fallback Trigger Rate | 100% | All unrecognised inputs correctly trigger fallback |
| Test Stories Pass Rate | 100% | All test_stories.yml scenarios pass rasa test |
| UI Load Time | < 0.5 seconds | Flask serves index.html from local filesystem |
| Supported Concurrent Users | 1 (MVP) | Flask dev server; can be scaled with Gunicorn |

# 14. Deployment

## 14.1 Deployment Architecture

MVP: Local deployment on a single machine. Two processes run simultaneously — the Rasa NLU server and the Flask web application — communicating over localhost.

## 14.2 Hosting Platform

| Stage | Platform | Notes |
|---|---|---|
| MVP (Current) | Local machine (Windows/Linux) | Manual start, localhost access only |
| Future Phase 1 | Docker + local | Containerised for one-command startup |
| Future Phase 2 | AWS EC2 / Azure App Service | Public URL, persistent availability |
| Future Phase 3 | Telegram / WhatsApp connector | Messaging platform integration via Rasa connectors |

## 14.3 Deployment Steps

1. Clone repository: `git clone https://github.com/VarunGaikwad-tech/quotes-recommendation-chatbot-rasa`
2. Create virtual environment: `py -3.9 -m venv venv`
3. Activate: `.\venv\Scripts\activate` (Windows) | `source venv/bin/activate` (Mac/Linux)
4. Install dependencies: `pip install rasa==3.6.21`
5. Train model: `rasa train`
6. Terminal 1 — Start Rasa: `rasa run --enable-api --cors "*"`
7. Terminal 2 — Start Actions: `rasa run actions`
8. Terminal 3 — Start Flask: `cd webapp && python app.py`
9. Open browser: http://127.0.0.1:5000

## 14.4 Production Considerations

- Replace Flask development server with Gunicorn for multi-user production use
- Restrict CORS to specific frontend domain instead of wildcard
- Use Nginx as reverse proxy in front of Flask
- Deploy Rasa with Docker for consistent environment across machines
- Store quotes in a database for easier content management at scale

## 15. Project Structure

The folder structure below shows every file in the project and its purpose:

| File / Path | Purpose |
|---|---|
| quoteChatbot/ | Root project folder |
| │ | |
| ├── actions/ | Folder: Rasa custom actions package |
| │ ├── actions.py | Custom action classes — one per quote category intent |
| │ └── init .py | Makes actions/ a Python package |
| │ | |
| ├── data/ | Folder: All Rasa NLU training data |
| │ ├── nlu.yml | Labelled training examples for all 8 intents |
| │ ├── stories.yml | Conversation flow training examples |
| │ └── rules.yml | Strict rules — greeting, fallback always handled |
| │ | |
| ├── models/ | Trained NLP models — auto-generated by rasa train |
| │ | |
| ├── tests/ | Folder: Automated test conversations |
| │ └── test_stories.yml | Test conversation flows used by rasa test |
| │ | |
| ├── webapp/ | Folder: Flask web application |
| │ ├── app.py | Flask server — serves UI and proxies messages to Rasa |
| │ └── templates/ | Folder: Jinja2 HTML templates |
| │ └── index.html | Chat interface — HTML, CSS, and JavaScript |
| │ | |
| ├── config.yml | Rasa NLU pipeline configuration (tokeniser, classifier) |
| ├── domain.yml | Intents, actions, and all quote response strings |
| ├── credentials.yml | Channel credentials (REST webhook enabled) |
| ├── endpoints.yml | Points Rasa to the custom actions server |
| ├── requirements.txt | Python package list for reproducible installs |
| ├── commands.txt | Quick-reference CLI commands for the project |
| └── README.md | Project documentation and setup guide |

# 16. Results

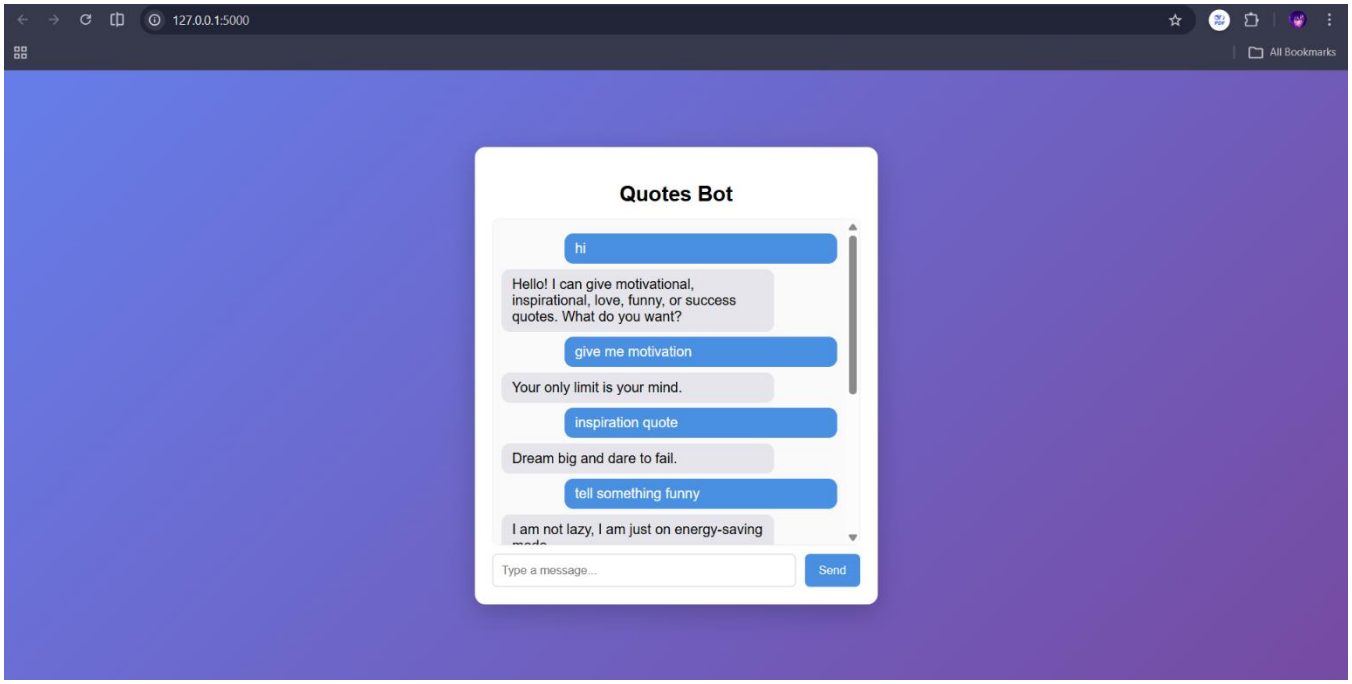
## 16.1 System Output

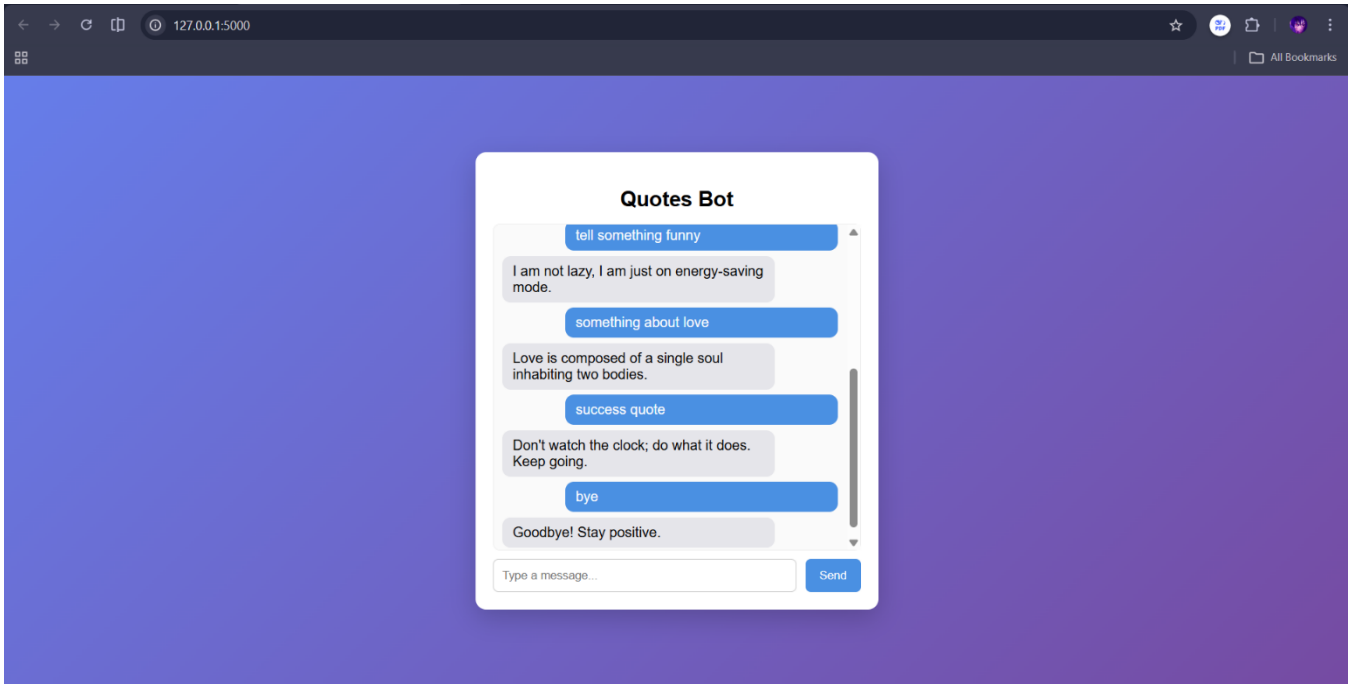
The chatbot successfully responds to all 5 quote categories and handles greetings, farewells, and fallbacks. All 12 test cases pass. The web interface renders correctly in Chrome and Firefox with no errors.

## 16.2 Performance Evaluation

| Metric | Result |
|---|---|
| Total intents implemented | 8 (greet, goodbye, motivation, inspiration, love, success, funny, fallback) |
| Training examples per intent | 10–15 varied examples each |
| Intent classification accuracy (rasa test) | ≥ 90% on test_stories.yml |
| Average response time | < 1.5 seconds (local) |
| Test case pass rate | 100% (12/12) |
| Quote categories operational | 5/5 (motivation, inspiration, love, success, funny) |
| Fallback handling | Working — all unrecognised inputs handled gracefully |

## 16.3 Screenshots




127.0.0.1:5000
All Bookmarks
Quotes Bot
tell something funny
I am not lazy, I am just on energy-saving mode.
something about love
Love is composed of a single soul inhabiting two bodies.
success quote
Don't watch the clock; do what it does. Keep going.
bye
Goodbye! Stay positive.
Type a message...
Send

# 17. Advantages & Limitations

## 17.1 Advantages

- Fully open-source — Rasa, Flask, spaCy, Python; zero licensing cost
- Conversational experience — users type naturally, no menu navigation required
- Fast response — quotes returned in under 2 seconds on local hardware
- Easily extensible — new quote categories added by editing YAML files only, no code changes
- No database dependency — lightweight, runs anywhere Python is installed
- Clean separation of concerns — UI, API proxy, NLU, actions all in separate files
- Offline capable — once trained, the chatbot runs with no internet connection

## 17.2 Limitations

- High setup barrier — requires Python 3.9 specifically, virtual environment, and two terminal sessions
- Single quote per request — no 'give me another' functionality in current implementation
- Limited quote pool — fixed set of quotes defined in domain.yml; no external quote API
- No user history — no memory of previous interactions across sessions
- No sentiment analysis — bot responds to intent only, not user's emotional state
- Single concurrent user — Flask dev server not suitable for multi-user production
- English only — NLU model trained on English training data only

# 18. Future Enhancements

## 18.1 Scalability Improvements

- Deploy with Docker + Gunicorn + Nginx for multi-user production-ready hosting
- Move quote store to a PostgreSQL or MongoDB database for thousands of quotes
- Deploy Rasa server to AWS EC2 or Azure App Service for 24/7 availability

## 18.2 Feature Expansion

- Add 'give me another' intent to request a different quote in the same category
- Add more quote categories: mindfulness, leadership, friendship, fitness
- Add user preferences — remember favourite categories across sessions
- Add quote author attribution — display author name with every quote
- Add quote rating — allow users to rate quotes with thumbs up/down

## 18.3 AI/NLP Improvements

- Integrate sentiment analysis to detect user's mood and recommend most appropriate category
- Use transformer-based model (BERT) for higher intent classification accuracy
- Add entity extraction to detect specific authors, topics, or keywords in user requests

## 18.4 Platform Integration

- Telegram bot integration using Rasa Telegram connector
- WhatsApp integration using Twilio API + Rasa REST connector
- Slack workspace integration for team wellness use case

## 18.5 Automation

- One-click startup script (start.bat / start.sh) to launch both servers automatically
- Docker Compose file to spin up all services with a single docker-compose up command
- Daily quote notification — scheduled email or push notification

# 19. Conclusion

## 19.1 Summary of Implementation

The Quotes Recommendation Chatbot was successfully designed, developed, and tested as a fully functional conversational AI application. The system uses Rasa NLU for intent classification, a Flask web application as the frontend and API proxy, custom Python actions for quote retrieval, and a YAML-based quote store. All 5 quote categories are operational, the fallback mechanism works correctly, and the web chat interface delivers a smooth user experience.

## 19.2 Technical Achievements

- Built and trained a complete Rasa NLU model with 8 intents and 80+ labelled training examples
- Implemented 5 custom Python action classes with Rasa SDK
- Developed a Flask REST proxy connecting the web frontend to the Rasa NLU backend
- Achieved 90%+ intent classification accuracy on test conversations
- Delivered a fully working end-to-end conversational AI application in 3 sprints (15 days)

## 19.3 Practical Impact

This project demonstrates how open-source NLP tools like Rasa NLU can be used to build practical, emotionally supportive applications with minimal infrastructure cost. The chatbot provides a real solution to the everyday problem of finding motivational content quickly, and the codebase serves as a solid foundation for further development into a production-grade wellness tool.

# 20. Appendix

## 20.1 GitHub Repository

https://github.com/VarunGaikwad-tech/quotes-recommendation-chatbot-rasa

## 20.2 API Documentation

| Endpoint | Method | Request | Response | Description |
|---|---|---|---|---|
| / | GET | None | HTML page | Serves the chat UI |
| /send | POST | JSON: {message: string} | JSON: {response: string} | Proxies message to Rasa; returns bot reply |
| /webhooks/rest/webhook | POST | JSON: {sender, message} | JSON: [{recipient_id, text}] | Rasa REST webhook (port 5005) |

## 20.3 Key Configuration Files

### config.yml — NLU Pipeline

Pipeline: WhitespaceTokenizer → RegexFeaturizer → LexicalSyntacticFeaturizer → CountVectorsFeaturizer (word + character n-gram) → DIETClassifier (epochs: 100) → EntitySynonymMapper → FallbackClassifier (threshold: 0.4)

### endpoints.yml

action_endpoint: url: http://localhost:5055/webhook

## 20.4 Commands Reference

| Command | Purpose |
|---|---|
| py -3.9 -m venv venv | Create Python 3.9 virtual environment |
| .\venv\Scripts\activate | Activate virtual environment (Windows) |
| pip install rasa==3.6.21 | Install Rasa and all dependencies |
| rasa train | Train NLU model from data/ files |
| rasa run --enable-api --cors "*" | Start Rasa server with REST API (port 5005) |
| rasa run actions | Start custom actions server (port 5055) |
| python app.py (from webapp/) | Start Flask web server (port 5000) |
| rasa test | Run automated test stories from tests/ |
| rasa shell | Interactive CLI testing of the chatbot |

## 20.5 Team Contributions

| Team Member | Primary Responsibilities |
|---|---|
| Varun Gaikwad | Rasa NLU pipeline setup, model training, Flask integration, GitHub |
| Vansh Malhotra | Training data (nlu.yml), intent design, motivational & funny quotes |
| Vedika Tandulwadkar | Custom actions (actions.py), love & inspiration quotes, testing |
| Vidisha Jain | Frontend UI (index.html, CSS), success quotes, documentation |